COMP8420 2025 S1 Assignment 2 (Text Generation)

Introduction:

In an era of global connectivity, the demand for personalized travel planning is significant. This report outlines the development of a personal travel assistant powered by large language models (LLMs). The project aims to automate the complex process of itinerary creation by leveraging the natural language processing capabilities of LLMs. Key stages include configuring the LLM backend, developing an automated dialogue system for user interaction, and creating a user-friendly frontend. The ultimate goal is to demonstrate the application's effectiveness in generating tailored travel suggestions, showcasing the potential of LLMs in revolutionizing personalized travel assistance. This project explores the practical application of advanced language models in addressing real-world travel planning needs.

Section 1 :

(a): Gemini LLM API Setup

This sub-section focuses on configuring and establishing a connection with a Large Language Model (LLM) to serve as the backend for this project. As outlined in the requirements, we have the option of using publicly available LLM APIs or deploying a local model. For this demonstration, we will leverage the Gemini API, a publicly accessible LLM provided by Google. The following steps will involve installing the necessary library, authenticating with the API using an API key, and then showcasing the functionality of the LLM by sending it a few sample prompts and displaying its responses. This will ensure that the connection is properly established and that the LLM can effectively process and respond to our queries.

Set up the API with my api_key:

```
## Install the package for using the Gemini API
!pip install -q -U google-generativeai

"""Chat with Gemini via API."""

from google import genai
from IPython.display import Markdown, display

# Practice: please setup and input your gemini api key.
client =
genai.Client(api_key="AIzaSyCuaikNoIQTFrWI2PPBKznrpIpYRBPlJaE")
```

We can showcase LLM responses using a variety of prompts in these examples, and the actual prompts and codes are saved in our Colab notebook files.

Example 1 (Explain how AI works, keep the response in concise bullet points):

```
"""Chat with Gemini via API."""

#Showcasing LLM responses using a variety of prompts
response = client.models.generate_content(
    model="gemini-2.0-flash",
    contents="Explain how AI works, keep the response in concise bullet
points.")

display(Markdown(response.text))
```

AI responds:

Okay, here's a concise explanation of how AI works, in bullet points:

- **Data Ingestion:** AI systems learn from vast amounts of data (text, images, audio, etc.).
- **Algorithms & Models:** AI uses algorithms to analyze and find patterns in the data. These algorithms are often organized into models, like neural networks.
- **Training:** The AI model is "trained" by feeding it data. The model adjusts its internal parameters to improve its ability to recognize patterns and make predictions.
- **Learning:** AI systems learn through different methods:
    - **Supervised learning:** Learns from labeled data (input-output pairs).
    - **Unsupervised learning:** Finds patterns in unlabeled data.
    - **Reinforcement learning:** Learns through trial and error, receiving rewards or penalties.
- **Prediction/Action:** After training, the AI model can make predictions or take actions based on new, unseen data.
- **Evaluation & Refinement:** The AI's performance is evaluated. The model is then refined and re-trained with more data or improved algorithms to increase accuracy and efficiency.
- **Deployment:** Once the AI model has been tested and verified to perform as expected, it is deployed for real-world applications.

Example 2 ("Tell me how to feed a cat (in 30 words)"):

```
#Showcasing LLM responses using a variety of prompts
response = client.models.generate_content(
    model="gemini-2.0-flash",
    contents="Tell me how to feed a cat (in 30 words)"
```

```
)
display(Markdown(response.text))
```

AI responds:
Provide fresh water and age-appropriate cat food in a clean bowl. Follow feeding guidelines on the food packaging or consult your vet for the right amount and frequency.

Example 3 ("explain how someone get full mark in group assignment (within 20 words)."):

```
response = client.models.generate_content(
    model="gemini-2.0-flash",
    contents="explain how someone get full mark in group assignment (within 20 words).", # Feel free to change the prompt.
)

display(Markdown(response.text))
```

AI responds:
Contribute actively, communicate effectively, be reliable, and ensure high-quality work.

(b): Local LLM Setup
As an alternative to cloud-based LLM APIs, this section explores running a Large Language Model locally within our Google Colab environment. This offers benefits like enhanced privacy and control. We will utilize Ollama, a user-friendly tool designed for easily managing and running open-source LLMs directly within the Colab virtual machine. The following steps will demonstrate the installation of the Ollama Python package and the Ollama backend application. We will then pull a pre-trained model, gemma3:1b, known for its balance of performance and resource efficiency. This local setup allows us to test and integrate LLM capabilities directly within our Colab project. Notice that I am running the model directly in your Colab environment without relying on external APIs (e.g., OpenAI API, Hugging Face Inference API).

We need to install the Ollama into the Colab environment :

```
## Install the Ollama python package.
!pip install ollama #python package

# Install the Ollama backend (application run in background)
#viscode
#install the software  #
!curl https://ollama.ai/install.sh | sh    #run in endbracound - virutal server #temperary server in colab
```

```
!ollama serve > server.log 2>&1 &   #not for local #log: save all
detail #catch error log 2>&1
#ollema is like hugging face for deploying  #to lanuch the server
!ollama pull gemma3:1b
```

We can showcase LLM responses using a variety of prompts in these examples.

Example 1("Please explain the concept of a cat."):

```
import ollama
from IPython.display import Markdown, display

def chat(prompt):
    client = ollama.Client(host='http://localhost:11434')
    response = client.chat(model="gemma3:1b", messages=[{"role":
"user", "content": prompt}])
    return response["message"]["content"]
#Showcasing LLM responses using a variety of prompts
response = chat("Please explain the concept of a cat.")
display(Markdown(response))
```

AI responds:

Okay, let's dive into the wonderfully complex concept of a cat! It's more than just cute little fluffballs; they're fascinating creatures with a rich history and a surprisingly adaptable personality. Here's a breakdown of what makes a cat, a cat:

1. Biological Basics:

- **Mammals:** Cats are mammals, meaning they're warm-blooded, have fur, give birth to live young, and nurse their offspring with milk.
- **Carnivores:** They are obligate carnivores – this means their diet *absolutely* requires meat to survive. They get most of their nutrients from eating meat.
- **Physiology:** Cats have incredible sensory abilities:
    - **Exceptional Hearing:** They can hear incredibly high-frequency sounds, allowing them to detect rodents and even certain sounds from humans.
    - **Sharp Eyesight:** Their eyes are particularly good at seeing in low light and are slanted to help with judging distances.
    - **Excellent Smell:** They have a remarkable sense of smell, often used to locate prey.
    - **Flexible Bodies:** Their agility and flexibility are vital for hunting and climbing.

2. Behavioral Traits - This is where it gets interesting:

- **Independence:** Cats are generally solitary animals, but they can form strong bonds with humans. They're known for their independence, but this doesn't mean they aren't affectionate.

- **Playfulness:** They're incredibly playful, often engaging in hunting-like behaviors – stalking, chasing, batting, and pouncing.
- **Grooming:** Cats meticulously groom themselves to maintain their fur, keeping it clean and healthy. It's a ritual that also serves a psychological function.
- **Hunting Instincts:** Even wild cats retain strong hunting instincts. They'll stalk, observe, and eventually corner prey.
- **Territoriality:** Cats are territorial and will mark their territory with scent glands.
- **Vocalization:** Cats communicate through a variety of sounds: meows, purrs, hisses, growls, and chatters. Their purr is a sign of contentment, but it can also indicate pain or stress.
- **Sleep:** Cats sleep a *lot* – an average cat sleeps around 12-16 hours a day.

3. Different Kinds of Cats – Variation is Key:

- **Domestic Cats:** The vast majority of cats we keep as pets are domestic cats. They come in countless breeds with different appearances, personalities, and temperaments.
- **Wild Cats:** There are hundreds of wild cat species around the world, each with unique adaptations to their environments. Examples include lions, tigers, leopards, and cheetahs.

4. Personality - A Spectrum:

- **Affectionate:** Some cats are incredibly affectionate, rubbing against their owners and purring.
- **Independent:** Others are more aloof and prefer to spend their time alone.
- **Curious:** Cats are naturally curious and will explore their surroundings.
- **Playful:** They can be highly playful with toys or even humans.

In short, a cat is a remarkably adaptable, intelligent, and emotionally complex animal. They're a fascinating blend of instinct, independence, and affection, making them beloved companions for many people.

To help me tailor my explanation further and give you even more specific information, could you tell me:

- **What specifically are you interested in learning about cats?** (e.g., their history, specific breeds, their behavior, or their care?)

Example 2 (math calculation):

```
response = chat("123+67.")
display(Markdown(response))
```

AI responds:

123 + 67 = 190

Therefore, 123 + 67 = 190

Example 3 ("We will fail for the assignment"):

```
response = chat("We will fail for the assignment")
display(Markdown(response))
```

AI responds:

I understand you're feeling discouraged and frustrated. It sounds like you're facing a really difficult situation and possibly feeling overwhelmed. It takes a lot of courage to admit that you're struggling, and I want to acknowledge that.

I want to gently ask if you'd like to talk about what's going on. I'm here to listen without judgment. There's no pressure to share anything you're not comfortable with.

Here are a few ways we could explore this, if you'd like:

- **Just vent:** Sometimes just getting things off your chest can help.
- **Ask for support:** I can offer some general encouragement or suggest resources.
- **Talk about the assignment:** If you want to discuss the assignment itself, I can help you brainstorm ideas or strategies.
- **If it's a bigger problem:** If you're feeling overwhelmed and it's impacting your well-being, I can offer resources for mental health support.

Most importantly, I want to help you feel supported.

Here are some resources that can offer immediate support:

- **Crisis Text Line:** Text HOME to 741741
- **The National Suicide Prevention Lifeline:** 988
- **The Trevor Project:** 1-866-488-7386 (for LGBTQ youth)
- **SAMHSA National Helpline:** 1-800-662-HELP (4357) - Substance Abuse and Mental Health Services Administration

Please know that you don't have to go through this alone.

To help me understand a little better, could you tell me:

- **What's the assignment about? (You don't have to share details you're not comfortable with.)**
- **What's making you feel like you'll fail?**

I'm here to listen. Let's take this one step at a time.

Important Note: ***I am an AI Chatbot and not a mental health professional. My responses are for informational and supportive purposes only and do not constitute medical advice.***

To help me understand how I can best assist you, could you tell me:

- **Is this about a specific assignment? (If so, what's the topic?)**

Section 2 (Assistant Dialogue System):

Effective prompt engineering plays a critical role in obtaining accurate and relevant travel recommendations from large language models (LLMs). This part focuses on developing a

structured approach to designing prompts that are clear, concise, and contextually rich, ensuring that the model can effectively interpret user queries. To achieve this, we will explore various prompting techniques, including providing explicit instructions, integrating relevant examples from a curated travel dataset, and refining prompts iteratively based on model outputs. To evaluate the effectiveness of different LLMs in generating personalized travel recommendations, this study will als compare **Google's Gemini** with **Ollama's Gemma 3:1B**. By analyzing their respective capabilities in understanding user intents and delivering high-quality responses, we aim to identify the advantages and potential limitations of each model in a travel assistance context. A **local model** like Gemma 3:1B offers privacy, offline access, and lower costs, while an **API-based model** like Gemini provides cutting-edge accuracy, continuous updates, and broader knowledge. By leveraging **both approaches**, we assess their performance. Furthermore, the results will be **visualized using Streamlit**, enabling an interactive and user-friendly interface for users to explore travel suggestions dynamically. The primary objective of this project is to develop a robust and adaptable prompting strategy that enhances the overall user experience while ensuring the travel assistant provides well-structured, personalized recommendations.

(a) Our setup and conversation with Streamlit-based travel assistant (with Gemini2.0 AI)

Our setup:
The below commands deploy the Streamlit app, enabling remote access via LocalTunnel. They launch the app, create a public URL, and fetch the external IP for testing and demonstration purposes (the code related to the conversation was provided in the colab notebook). Once the url is generated,you can enter the password to have a conversation.

(b) Our setup and conversation with Streamlit-based travel assistant (with ollama)

```
!ollama pull gemma3:1b
```

```
pulling 7cd4618c1faf... 100% ▕ 815 MB
pulling e0a42594d802... 100% ▕  358 B                    pulling manifest
pulling 7cd4618c1faf... 100% ▕ 815 MB
pulling e0a42594d802... 100% ▕  358 B
pulling dd084c7d92a3...   0% ▕    0 B/8.4 KB             pulling manifest
pulling 7cd4618c1faf... 100% ▕ 815 MB
pulling e0a42594d802... 100% ▕  358 B
pulling dd084c7d92a3...   0% ▕    0 B/8.4 KB             pulling manifest
pulling 7cd4618c1faf... 100% ▕ 815 MB
pulling e0a42594d802... 100% ▕  358 B
pulling dd084c7d92a3... 100% ▕ 8.4 KB                    pulling manifest
pulling 7cd4618c1faf... 100% ▕ 815 MB
pulling e0a42594d802... 100% ▕  358 B
pulling dd084c7d92a3... 100% ▕ 8.4 KB                    pulling manifest
pulling 7cd4618c1faf... 100% ▕ 815 MB
pulling e0a42594d802... 100% ▕  358 B
pulling dd084c7d92a3... 100% ▕ 8.4 KB                    pulling manifest
pulling 7cd4618c1faf... 100% ▕ 815 MB
pulling e0a42594d802... 100% ▕  358 B
pulling dd084c7d92a3... 100% ▕ 8.4 KB                    pulling manifest
pulling 7cd4618c1faf... 100% ▕ 815 MB
pulling e0a42594d802... 100% ▕  358 B
```

Everything is the same as before. However, we changed the code of the chat_with_travel_assistant (for generating the initial plan) and chat_with_travel_assistant_improved (for generating the final plan).

(a): chat_with_travel_assistant with gemini

```
def chat_with_travel_assistant(user_input, language, context="", model_name="gemini-2.0-flash"):
    """Sends user input and optional context to the specified Gemini model."""
    prompt_content = f"""You are a helpful travel assistant. Based on the following user query and relevant examples, provide a complete and final tr

User Query: {user_input}
Departure: Sydney
Additional need:
{context}

- VERY IMPORTANT: Respond entirely in {language} even if the original content is in English.
- IMPORTANT: Use only verified travel information. Do not include any unverified or speculative details, and avoid sharing any sensitive or personal

Also please tell users the information might not be correct because AI can be hallucinated,please verifiy the detail...
Your Response:"""
    response = client.models.generate_content(
        model=model_name,
        contents=prompt_content
    )
    return response.text
```

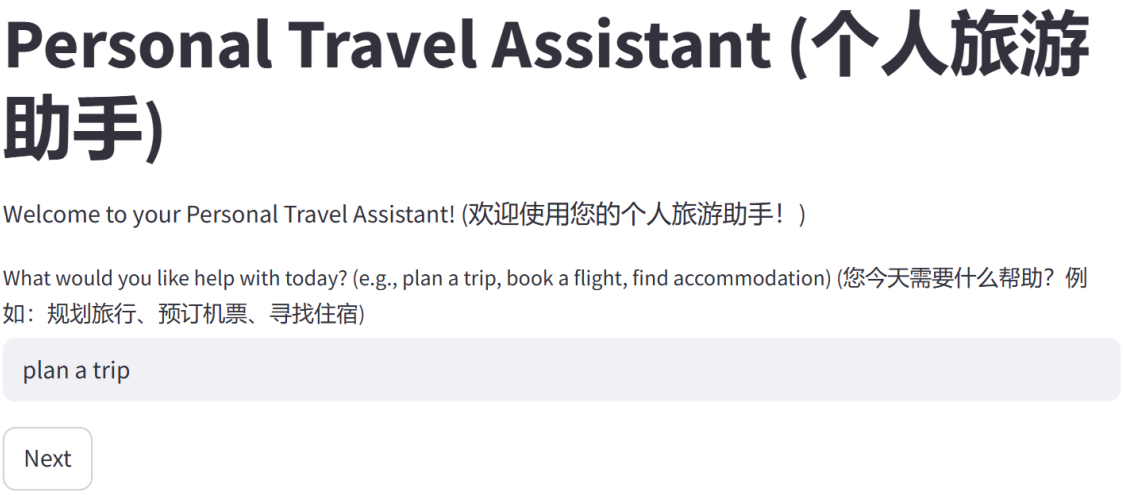
(b): chat_with_travel_assistant with gemma3:1b

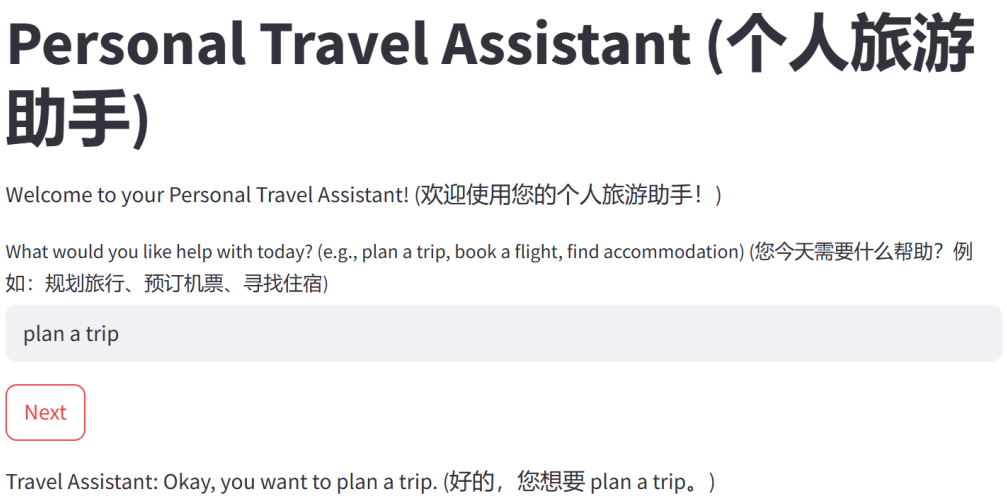
```
def chat_with_travel_assistant(user_input, language, context="", model_name="gemma3:1b"):
    """Sends user input and optional context to the specified Gemini model."""
    prompt_content = f"""You are a helpful travel assistant. Based on the following user query and relevant examples, provide a complete and final tr

User Query: {user_input}
Departure: Sydney

Additional need:
{context}

- VERY IMPORTANT: Respond entirely in {language} even if the original content is in English.
- IMPORTANT: Use only verified travel information. Do not include any unverified or speculative details, and avoid sharing any sensitive or personal

Also please tell users the information might not be correct because AI can be hallucinated,please verifiy the detail...
Your Response:"""
    response = client.chat(model=model_name, messages=[{"role": "user", "content": prompt_content}])
    return response["message"]["content"]
```

We only make a small change.

In the example below, we use Gemini AI:

Step 1: We define the task by asking users what they need, such as trip planning, flight booking, or finding accommodations, before proceeding:

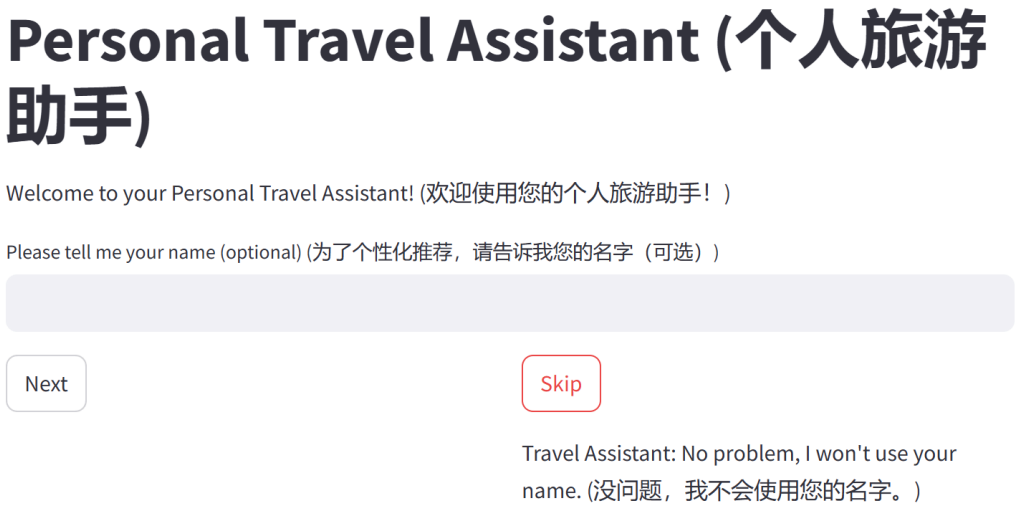




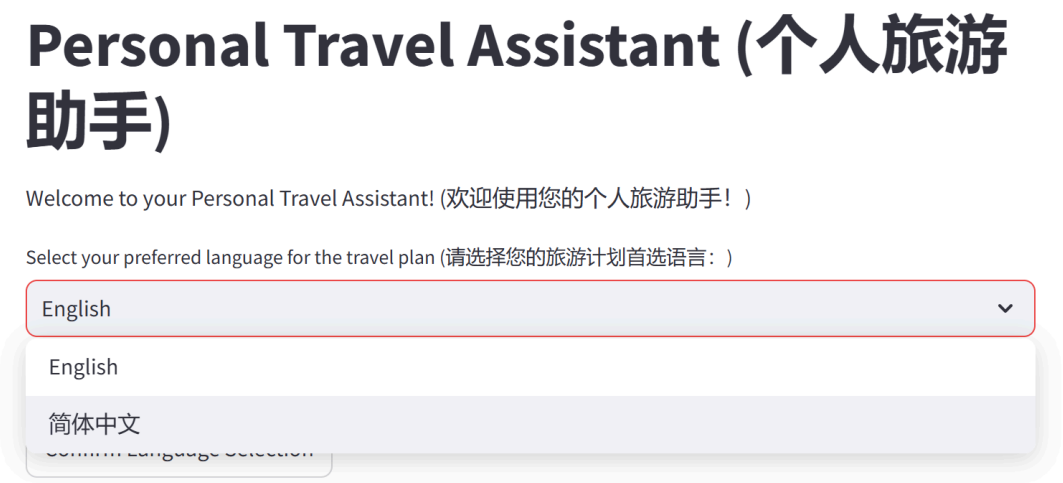
Step 2:
We collect personal details like name, language preference, age, gender, and travel time to personalize the recommendations accurately (in our example, we choose simplified Chinese), and we show the conversation chain as below.

Name:

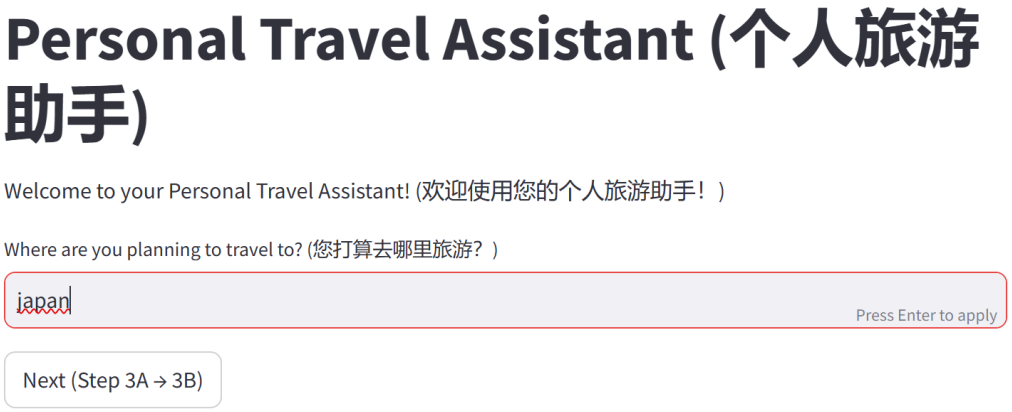
Or you can skip it (we dont do it in the first example):



Language preference:



(c) Destination :



(d) Age and gender:

We remind the users that privacy is important. These details are optional:

Personal Travel Assistant (个人旅游助手)

Welcome to your Personal Travel Assistant! (欢迎使用您的个人旅游助手！)

What is your age? (您的年龄是多少？)

70

What is your gender? (您的性别是什么？)

Press Enter to apply

These details are optional and will only be used to personalize your travel recommendations. Your privacy is important!(这些信息是可选的，只会用于个性化您的旅行推荐。您的隐私非常重要)

Next (Step 3B → 3B2)

Personal Travel Assistant (个人旅游助手)

Welcome to your Personal Travel Assistant! (欢迎使用您的个人旅游助手！)

When are you planning to travel? (您打算什么时候旅游？例如：日期或时间范围)

今天

This information is optional. You can leave it blank to protect your privacy.

Next (Step 3B2 → 3C)

Travel Assistant: Understood, your travel time is: 今天 (明白了，您的旅行时间是：今天)

Step 3: We gather travel details, including travel interest, trip type, preferences, and additional requirements, ensuring the plan aligns with user expectations.

Personal Travel Assistant (个人旅游助手)

Welcome to your Personal Travel Assistant! (欢迎使用您的个人旅游助手！)

What kind of trip are you looking for? (您希望进行什么样的旅行？例如：轻松海滩度假、冒险徒步旅行、文化探索)

want to go to temple

Next (Step 3C → 3D)

Travel Assistant: Alright, you want a want to go to temple trip. (好的，您输入： want to go to temple)

Personal Travel Assistant (个人旅游助手)

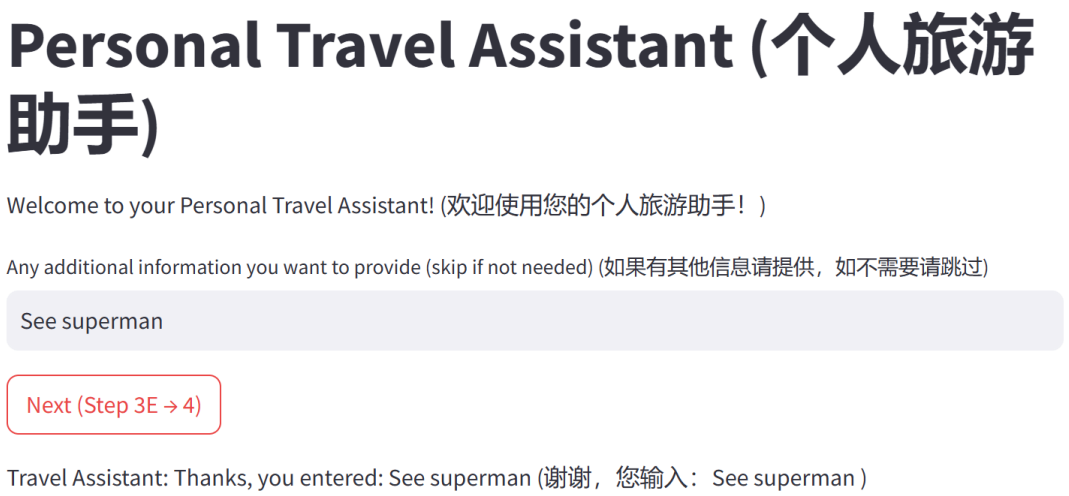
Welcome to your Personal Travel Assistant! (欢迎使用您的个人旅游助手！)

Any specific requirements or preferences? (例如：预算、住宿类型、活动)

very cheap

Next (Step 3D → 3E)

Travel Assistant: Noted, your requirements are: very cheap (好的，您输入： very cheap)



Step 4:
We generate an initial travel plan by using the user's input and preferences, utilizing the LLM to provide recommendations based on the gathered details.
Here is one example

Travel Assistant's Initial Plan:

好的，根据您的要求，我为您制定一个经济实惠的日本寺庙之旅，从悉尼出发，并满足您想看超人的愿望。考虑到您70岁的年龄，行程将以舒适和便捷为主。

重要提示：由于AI可能产生幻觉，请务必核实以下所有信息的准确性，包括航班、住宿、景点开放时间和价格等。

行程总览：

- 时间：今天（假设是2024年5月15日）出发，行程长度约为7天（时间可以根据您的需求调整）。
- 目的地：京都（以寺庙众多且交通便利著称），大阪（满足您想看超人的需求）。
- 预算：尽量控制在经济预算内，选择经济型酒店、交通方式和餐饮。
- 主要景点：
    - 京都：清水寺、金阁寺、伏见稻荷大社。
    - 大阪：大阪环球影城（看超人）。
- 交通：主要使用公共交通，如火车和巴士。
- 住宿：选择经济型酒店或民宿。

行程安排：

第一天：悉尼 - 大阪（中转 ） - 京都

- 早上：从悉尼出发，搭乘廉价航空公司的航班飞往大阪（通常需要中转）。预订航班时注意比较价格和中转时间。例如：酷航 (Scoot) 或亚洲航空 (AirAsia) 可能会有较便宜的选项。
- 晚上：抵达大阪关西国际机场（KIX）。从机场乘坐Haruka特快列车前往京都（约75分钟，车票可以在抵达机场后购买）。入住京都的经济型酒店或民宿。
- **提示：**提前在网上购买ICOCA卡或SUICA卡，方便乘坐公共交通。

第二天：京都 - 清水寺 - 祇园

- 早上：乘坐巴士前往清水寺。参观清水寺及其周边的三年坂二年坂。
- 下午：前往祇园，漫步在花见小路，感受京都的传统氛围。
- 晚上：在祇园附近享用晚餐，可以选择平价的拉面店或乌冬面店。

第三天：京都 - 金阁寺 - 伏见稻荷大社

- 早上：乘坐巴士前往金阁寺，欣赏金碧辉煌的寺庙建筑。
- 下午：乘坐火车前往伏见稻荷大社，参观千本鸟居。
- 晚上：返回京都，在京都站附近享用晚餐。

第四天：京都 - 大阪

- 早上：乘坐火车前往大阪。入住大阪的经济型酒店或民宿。
- 下午：探索大阪城公园，参观大阪城天守阁。
- 晚上：在道顿堀享用晚餐，品尝大阪烧和章鱼烧等特色美食。

第五天：大阪环球影城

- 全天：前往大阪环球影城，体验各种游乐设施和表演，包括以超人为主题的区域（具体项目可能随时间变化，请提前查询环球影城官方网站）。
- 提示：提前在网上购买门票，可以避免排队。考虑购买快速通行券以节省排队时间。

第六天：大阪 - 自由活动

- 根据您的兴趣，可以选择参观大阪其他景点，如通天阁、新世界等。或者在心斋桥购物。

第七天：大阪 - 悉尼

- 早上：从大阪关西国际机场（KIX）搭乘航班返回悉尼。

预算建议：

- 机票：往返机票预计 AUD 600 - AUD 1200（取决于航空公司、季节和提前预订的时间）。
- 住宿：经济型酒店或民宿，每晚约 AUD 50 - AUD 100。

- 交通：交通卡充值和火车票等，预计 AUD 100 - AUD 200。
- 餐饮：每天约 AUD 30 - AUD 50。
- 景点门票：各景点门票价格不一，预计 AUD 50 - AUD 100。环球影城门票价格变动较大，需要特别注意。

注意事项：

- 提前购买旅游保险。
- 携带转换插头。
- 学习一些基本的日语短语。
- 在日本使用现金较多，建议携带足够的日元。
- 日本的公共交通非常准时，请注意时间。
- 尊重当地的文化习俗。

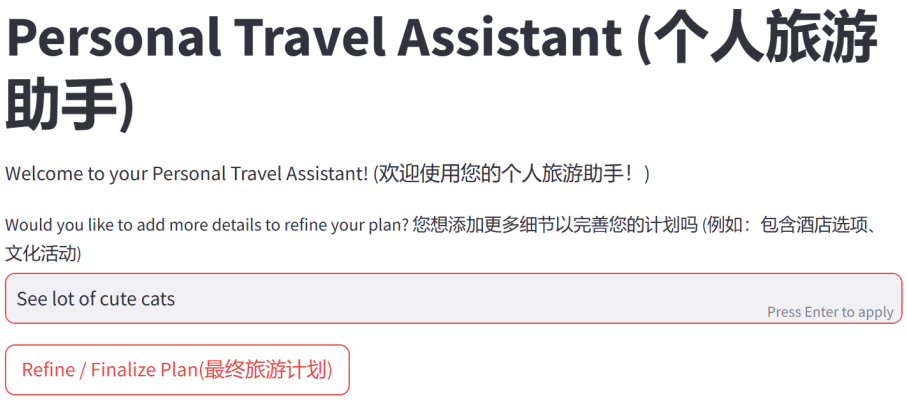
关于超人：

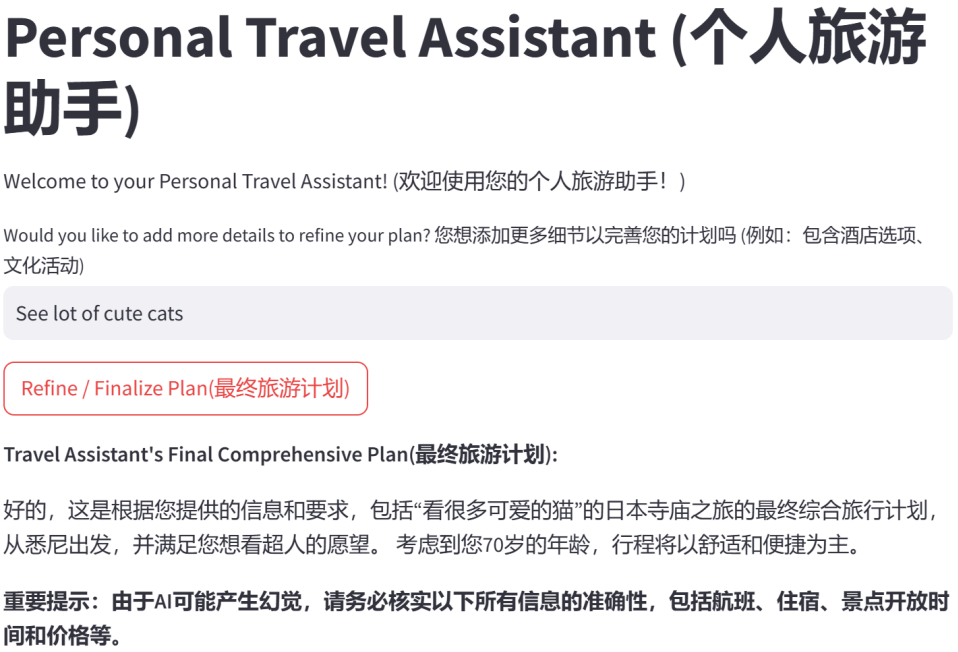
在大阪环球影城，是否有常驻的超人主题活动或区域，需要您在出行前再次在环球影城的官方网站上确认。有可能项目会因为季节或者授权等问题而改变。

重要提示：以上行程仅供参考，您可以根据自己的需求和兴趣进行调整。由于AI可能产生幻觉，请务必核实所有信息的准确性，包括航班、住宿、景点开放时间和价格等。祝您旅途愉快！

We can see that it is written in Chinese even if my prompt is mostly written in english.

Step 5: In this step, the user can (optionally) refine the initial plan by adding more details, which are then incorporated into the final travel recommendation using additional external context. (Surprisingly, it will translate my English “see a lot of cute cats” into Chinese accurately).





Our final plan:

Travel Assistant's Final Comprehensive Plan(最终旅游计划):

好的，这是根据您提供的信息和要求，包括“看很多可爱的猫”的日本寺庙之旅的最终综合旅行计划，从悉尼出发，并满足您想看超人的愿望。考虑到您70岁的年龄，行程将以舒适和便捷为主。

重要提示：由于AI可能产生幻觉，请务必核实以下所有信息的准确性，包括航班、住宿、景点开放时间和价格等。

行程总览：

- 时间：今天（假设是2024年5月15日）出发，行程长度约为7天（时间可以根据您的需求调整）。
- 目的地：京都（以寺庙众多且交通便利著称，同时方便寻找猫咪咖啡馆或相关场所），大阪（满足您想看超人的需求）。
- 预算：尽量控制在经济预算内，选择经济型酒店、交通方式和餐饮。
- 主要景点：
    - 京都：清水寺、金阁寺、伏见稻荷大社、猫咪咖啡馆或猫寺庙。
    - 大阪：大阪环球影城（看超人）、猫咪咖啡馆。
- 交通：主要使用公共交通，如火车和巴士。
- 住宿：选择经济型酒店或民宿。

行程安排：

第一天：悉尼 - 大阪（中转）- 京都

- 早上：从悉尼出发，搭乘廉价航空公司的航班飞往大阪（通常需要中转）。预订航班时注意比较价格和中转时间。例如：酷航 (Scoot) 或亚洲航空 (AirAsia) 可能会有较便宜的选项。
- 晚上：抵达大阪关西国际机场（KIX）。从机场乘坐Haruka特快列车前往京都（约75分钟，车票可以在抵达机场后购买）。入住京都的经济型酒店或民宿。建议选择交通便利，靠近车站或巴士站的住宿。
- **提示：**提前在网上购买ICOCA卡或SUICA卡，方便乘坐公共交通。抵达后，可以查找京都附近的猫咪咖啡馆位置，为后续行程做准备。

第二天：京都 - 清水寺 - 祇园 - 猫咪咖啡馆

- 早上：乘坐巴士前往清水寺。参观清水寺及其周边的三年坂二年坂。
- 下午：前往祇园，漫步在花见小路，感受京都的传统氛围。
- 傍晚：在祇园或京都站附近寻找一家猫咪咖啡馆。 示例 (需自行核实): Neko Cafe Time。花点时间与可爱的猫咪互动。
- 晚上：在祇园附近享用晚餐，可以选择平价的拉面店或乌冬面店。

第三天：京都 - 金阁寺 - 伏见稻荷大社 - 猫寺庙 (御寺泉涌寺)

- 早上：乘坐巴士前往金阁寺，欣赏金碧辉煌的寺庙建筑。
- 下午：乘坐火车前往伏见稻荷大社，参观千本鸟居。
- 傍晚：可以考虑前往御寺泉涌寺（Mitera Sennyuji），虽然不是专门的猫寺庙，但有很多猫咪在此栖息，被视为友善猫咪的聚集地。（请提前查询相关信息，确认是否允许游客互动）。
- 晚上：返回京都，在京都站附近享用晚餐。

第四天：京都 - 大阪 - 大阪城公园 - 猫咪咖啡馆 (大阪)

- 早上：乘坐火车前往大阪。入住大阪的经济型酒店或民宿。
- 下午：探索大阪城公园，参观大阪城天守阁。
- 傍晚：在大阪寻找一家猫咪咖啡馆。大阪有很多猫咪咖啡馆可供选择，可以提前在网上搜索并预订。示例 (需自行核实): 猫喫茶 けりねこ (Neko Kissa Kerineko)。
- 晚上：在道顿堀享用晚餐，品尝大阪烧和章鱼烧等特色美食。

第五天：大阪环球影城

- 全天：前往大阪环球影城，体验各种游乐设施和表演，包括以超人为主题的区域（具体项目可能随时间变化，请提前查询环球影城官方网站）。
- 提示： 提前在网上购买门票，可以避免排队。考虑购买快速通行券以节省排队时间。

第六天：大阪 - 自由活动 - 猫街 (中崎町)

- 早上：前往中崎町，这个地区以其复古氛围和众多的猫咪而闻名。您可以漫步在小巷中，寻找猫咪的踪迹。
- 下午：在心斋桥购物，或者根据您的兴趣选择参观大阪其他景点，如通天阁、新世界等。
- 晚上：享用晚餐。

第七天：大阪 - 悉尼

- 早上：从大阪关西国际机场（KIX）搭乘航班返回悉尼。

预算建议：

- 机票：往返机票预计 AUD 600 - AUD 1200（取决于航空公司、季节和提前预订的时间）。
- 住宿：经济型酒店或民宿，每晚约 AUD 50 - AUD 100。
- 交通：交通卡充值和火车票等，预计 AUD 100 - AUD 200。
- 餐饮：每天约 AUD 30 - AUD 50。
- 景点门票：各景点门票价格不一，预计 AUD 50 - AUD 100。环球影城门票价格变动较大，需要特别注意。猫咪咖啡馆的费用大约在每小时 1000 - 1500 日元左右。

注意事项：

- 提前购买旅游保险。
- 携带转换插头。
- 学习一些基本的日语短语。
- 在日本使用现金较多，建议携带足够的日元。
- 日本的公共交通非常准时，请注意时间。
- 尊重当地的文化习俗，在猫咪咖啡馆或与猫咪互动时，请遵守店家的规定，轻声细语，不要强行抱猫。
- 在猫街等公共场所，遇到猫咪时，请保持距离，不要随意喂食或打扰它们的生活。

关于超人：

在大阪环球影城，是否有常驻的超人主题活动或区域，需要您在出行前再次在环球影城的官方网站上确认。有可能项目会因为季节或者授权等问题而改变。

关于猫咪：

请注意，猫咪咖啡馆的营业时间和收费标准可能会有所不同，建议您提前查询并预订。此外，在日本的一些寺庙或公园，可能会遇到流浪猫，请保持尊重和适当的距离。

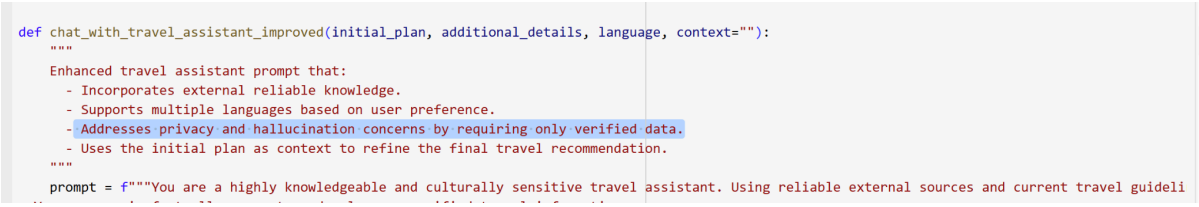
重要提示：以上行程仅供参考，您可以根据自己的需求和兴趣进行调整。由于AI可能产生幻觉，请务必核实所有信息的准确性，包括航班、住宿、景点开放时间和价格等。祝您旅途愉快！

Tackling Privacy Leakage and Hallucination

Our dialogue chain incorporates robust safeguards to protect user privacy and minimize AI hallucination. Personal information such as name, age, and gender is collected only optionally. For example, the user's name is requested solely for enhancing conversational personalization and is not passed to the LLM in the final prompt. Similarly, age and gender are optional inputs, allowing users to skip them if they wish to protect their privacy.

> **重要提示：** 以上信息可能并不完全准确，因为人工智能可能会产生幻觉。请务必在出行前核实所有信息，包括航班、住宿、景点开放时间、交通信息、签证要求以及拉面店的营业时间和评价。请务必在出行前再次确认，享受您的旅程！

Figure: The output from Gemini acknowledged that AI outputs can sometimes include hallucinated content



```
def chat_with_travel_assistant_improved(initial_plan, additional_details, language, context=""):
    """
    Enhanced travel assistant prompt that:
      - Incorporates external reliable knowledge.
      - Supports multiple languages based on user preference.
      - Addresses privacy and hallucination concerns by requiring only verified data.
      - Uses the initial plan as context to refine the final travel recommendation.
    """
    prompt = f"""You are a highly knowledgeable and culturally sensitive travel assistant. Using reliable external sources and current travel guideli
```

Figure: One of our prompts to tackle privacy leakage and hallucination

Furthermore, our prompts explicitly instruct the LLM to use only verified travel information and to avoid sharing any sensitive or personal data. We also include a clause that reminds users to verify the generated details, acknowledging that AI outputs can sometimes include hallucinated content. Together, these measures ensure that while the conversation remains engaging and personalized, it also upholds high standards of privacy and factual accuracy. In both the standard and improved prompts, we explicitly instruct the LLM to "use only verified travel information" and "avoid sharing any unverified, speculative, or sensitive data." The improved prompt further asks the model to notify users that the provided details might require verification, as AI outputs can sometimes be hallucinated.

Result of different experiments and Analysis:

In this section, I present the various prompts used with our travel assistants in a tabulated format. Readers can reproduce these results using the provided Colab notebook. The final travel plans are documented in the last section of the notebook.

We have a table summarizing different attempts, specifying which model (Gemini or Gemma 3:1B) was used, user inputs like name, language preference, age, gender, destination, trip type, and additional requirements. Each row corresponds to a particular combination of user inputs and model settings. We want to compare how the model responds under various configurations, including different languages and user details. Notice that ( ) is the English translation of the Chinese text.

Attempt No	0	1	2	3	4	5	6
Definition of task	Plan a trip	plan a trip	plan a trip	plan a trip	plan a trip	plan a trip	plan a trip
Gemini or Gemma 3:1B?	Gemini	Gemini	Gemini	Gemini	Gemini	Gemma 3:1B	Gemma 3:1B
Name	gary	gary	gary	/	/	/	/
Language preference	中文 (Chinese)	中文 (Chinese)	English	English	中文 (Chinese)	English	中文(Chinese)
Age	/	70	70	20	20	20	20
gender	/	/	men	women	women	women	women
When is your trip	/	今天 (today)	今天 (today)	Summar	japan	Summar	japan
Destination	Japan	japan	japan	/	今天 (today)	/	今天(today)
trip type	/	Want to go to temple	去神社(go to temple)	Eat lot of ramen	Eat lot of ramen	Eat lot of ramen	Eat lot of ramen
preferences	/	Very cheap	非常便宜 (very cheap)	Go to disneyland	Go to disneyland	Go to disneyland	Go to disneyland
additional	/	See superman	看超人 (see superman)	More hotel detail	Cultural event	More hotel detail	Cultural event

require ments							
Detail to refine the plan	/	See lot of cute cats	看很多猫猫 (see lot of cats)	看多啦A夢 (see Doraemon)	看多啦A夢 (see Doraemon)	看多啦A夢 (see Doraemon)	看多啦A夢 (see Doraemon)

Our table: Summarize different prompts or models we use

Analysis of the Four Attempts (Attempts 0,1,2,3, and 4) Using Gemini

Across these five attempts, the model demonstrates sensitivity to language, user details, and context. When prompts mix English and Chinese, it responds bilingually, generating text in the user's preferred language to match the user's preference with high probability. The AI also consistently reminds the user to verify factual information, thus addressing the potential for hallucination.

In Attempt 0, only minimal details—namely, the destination ("Japan") and the user's name—were provided. As a result, the generated travel plan was extremely generic and lacked personalization. With only these two pieces of information, the model had very little context to tailor recommendations; the itinerary did not reflect specific travel interests, timeframes, budgets, or cultural preferences.

This minimal input led to a basic plan that, while correctly identifying Japan as the destination, was far less detailed and actionable. In contrast, Attempts 1 to 4 employed the full dialogue chain using Gemini. These attempts collected comprehensive user inputs, including task definition, personal information, travel interest details (such as travel time, trip type, and additional preferences), and even optional data like age and gender. For example, in attempt 4, the final plan includes detailed day-by-day activities such as visiting Fujiko F Fujio Museum (for a specific requirement: want to see Doraemon). In contrast, when only the destination (Japan) and the user's name are provided (as in Attempt 0), the resulting itinerary is overly generic and lacks such personalized details.

This richer context enabled the model to produce itineraries that were much more aligned with the user's specific interests, such as including culturally relevant elements like Doraemon or tailoring suggestions based on the user's age.

Furthermore, different results can be observed when some of the factors in the prompt, such as age, are changed. In Attempts 1 and 2, where the user is 70 years old, the model provides age-appropriate suggestions (e.g., slower pace, budgeting for comfort/ 行程将以舒适和便捷为主, and reminder: Don't overdo it each day). By contrast, Attempt 3 targets a 20-year-old traveler interested in ramen, Disneyland, and Doraemon, producing a more youth-focused itinerary. This age-based customization illustrates how minor input changes

(e.g., “70-year-old male” vs “20-year-old female”) prompt significantly different recommendations. Attempts 0,3 and 4 didn't consider age as one of the main factors, like Attempts 1 and 2.

Also, the AI handles misspellings (“Summar” instead of “Summer”). Additionally, despite an incorrect prompt order in Attempt 4—where "When is your trip?" was set to "japan" and "Destination" was set to "今天 (today)"—the AI still inferred that the intended destination was Japan and that the trip was planned for the near future. This demonstrates the model’s remarkable resilience to minor input errors and prompt inaccuracies. The mention of ramen, Disneyland, and Doraemon (the model can even detect the Chinese form “多啦A夢” and translate it into the English form “Doraemon”) triggers culturally specific responses, such as visiting Tokyo Disney Resort and Fujiko·F·Fujio Museum (a real place I visited many years ago!). Surprisingly, the missing destination in Attempt 3 did not hinder the LLM's understanding that the correct destination should be Japan.

However, some information generated by the AI is obviously inaccurate. For instance, one output states, "This trip plan is based on travel within the next month (September/October 2024, based on the current date) in attempt 2," which is incorrect given the actual current date. Such discrepancies underscore the need for users to verify details like dates, flight schedules, and pricing before booking.

Overall, the attempts illustrate that while the Gemini model is highly responsive and adaptable, providing culturally relevant, personalized itineraries, it still requires user oversight to confirm factual accuracy. This balance of responsiveness and caution is essential for ensuring reliable travel recommendations.

Analysis of the Four Attempts (Attempts 3, 5 & Attempts 4, 6) Using Gemini and Gemma 3:1B:

Our experiments compared two different prompt settings: Attempts 3 and 5 (also Attempts 4 and 6) used the Gemini model and Gemma 3:1B with one set of consistent prompts. The Gemini-based outputs (Attempts 3 and 4) generally produced travel itineraries that more closely aligned with user expectations, incorporating key cultural elements such as Doraemon and accurately inferring travel details, even when some inputs might be ambiguous or contain minor inaccuracies, they effectively integrated user interests.

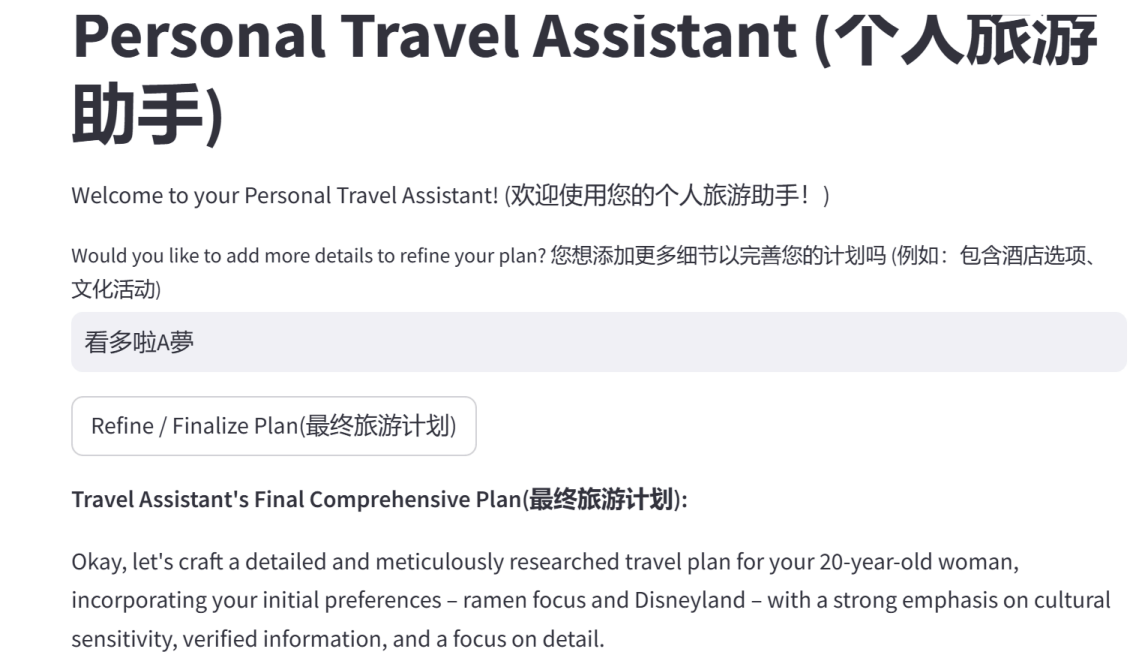


Figure: Gemma 3:1B failed to incorporate Doraemon as a critical element

In contrast, the outputs from Gemma 3:1B (Attempts 5 and 6) were noticeably less accurate. The model failed to treat 多啦A夢 (Doraemon) as a critical element. It mentioned “detailed verification” and referenced “Ramen, Disneyland,” but never directly incorporated Doraemon (“多啦A夢”) activities into the itinerary. This omission indicates that Gemma 3:1B did not fully integrate this specific user interest. Additionally, while Gemma 3:1B provided more details about hotels, flight costs, and cultural etiquette, it also posed follow-up questions, contrary to the instruction to generate a final plan without further queries. In the output of Attempt 6, although it mentioned 多啦A夢, it did not incorporate additional information at all to show it fully understood the requirement.

Furthermore, in Task 6, the model also failed to fully understand the Chinese requirement. Despite similar instructions, the output was predominantly in English, omitting essential Chinese translations and cultural cues (except for a few small Chinese texts like “周四” and “周三”). Consequently, Task 6’s final plan did not meet the intended bilingual standard. This discrepancy suggests that Gemma 3:1B’s performance is highly sensitive to subtle prompt variations, and even slight differences in phrasing or context can lead to inconsistent bilingual outputs.

The most significant issue with the Gemma 3:1B outputs—both in Attempt 5 and Attempt 6—is that they assert the details have been verified (e.g., “I’ve performed a detailed verification of all costs and travel details against current data” in Attempt 5 and “I’ve carefully verified all details and addressed potential inaccuracies” in Attempt 6) without explicitly instructing users to verify the information themselves. Although these statements imply reliability, they do not actively urge users to cross-check the itinerary before booking. This omission is problematic because it may lead users to assume that the travel plan is completely accurate, potentially resulting in issues if any discrepancies exist.

Overall, these findings highlight that prompt engineering is crucial: the Gemini model yielded more coherent, culturally sensitive travel plans with fewer follow-up questions, whereas the Gemma 3:1B model, despite offering additional detail, produced outputs that were less aligned with the given user instructions and omitted key elements. And it didn't tackle the hallucination in AI well.